TECHNICAL REPORT

FROM FOUNDATIONS TO EXPLOITATION

A Four-Week Security Journey

A Structured Reflection on Core Security Practice

Analyst Name: ONWUBIKO GODSWILL CHINAECHEREM	**Class Section:** Cybersecurity Class B
Registration No: tr3/std/cyb/099	**Program:** Abia TechRise Cybersecurity & DevSecOps (Cohort 3.0)
Date: June 25, 2026	**Classification:** TECHNICAL REPORT
Project Repository : **https://github.com/Godswill-star-spec/Onwubiko_Godswill_C--Techrise-weeks1to4.git**	

Section 1: Week 1 Foundation (How it connects to everything else)

Week 1 focused on basic Linux file system operations, navigation, and user permissions. These skills are essential because every major security platform, tool, and log repository relies on an administrative environment to operate. Understanding how to move through directories and inspect settings creates the direct ability to find, investigate, and secure system files in subsequent analysis phases. At first these exercises felt almost too simple to matter, but I quickly learned that nearly every later task in the programme, from scripting to scanning, assumes a working comfort with the command line that has to be built somewhere, and Week 1 is where that foundation gets laid.

Running the command **chmod 600** restricts access so that only the file owner can read and modify it. In a real investigation, this action is directly tied to protecting the chain of custody. By locking down raw log files or harvested evidence immediately upon discovery, an analyst prevents accidental changes or tampering by unauthorized users, ensuring the collected information remains legally valid and reliable for later compliance reporting. I found this connection genuinely useful: it is one thing to memorize a permission command, and another to understand that the same command stands between a piece of digital evidence and a courtroom or disciplinary hearing where its integrity might be questioned.

Using the command **ls -la** allows a user to see hidden files and complete details about permissions and modification dates. This shares the exact same objective as searching for backdoor scripts after an exploit occurs. Malicious actors frequently name their unauthorized scripts with leading dots or hide them inside common system folders. Being comfortable auditing basic file properties ensures you can easily spot unknown files or suspicious permission modifications later on. It is a small habit, checking timestamps and ownership almost automatically, but it is exactly the kind of habit that separates someone who reads about security from someone who actually practices it.

A clear example of this interaction occurs when handling any system file, such as the standard Linux authentication records. Without the basic ability to navigate administrative directories, find hidden files, and adjust permissions learned in Week 1, I as an analyst would be unable to open, extract, or parse data lines during automated scripting tasks or verify service properties during vulnerability assessments.

Section 2: Week 2 Intelligence (How knowing your enemy changes everything)

Week 2 focused on threat intelligence, open-source gathering **(OSINT- Open Source Intelligence)**, and risk modeling. Gathering strategic intelligence transforms security work from blindly checking configurations into targeted, structured defenses. Knowing the specific types of threats facing an environment determines exactly where defensive resources should be applied. This shift in thinking was important for me personally, because before this week I assumed security testing simply meant running a tool against a target and reading whatever came back. Week 2 reframed that assumption entirely, showing that effective testing actually begins long before any tool is opened, with research into who the likely attackers are and what they tend to look for.

Using **WHOIS** registries and DNS tools helps map out public network boundaries and connected assets. This research provides a list of valid IP ranges and active domain names that saves time during later assessment stages. Instead of scanning entire networks at random, an engineer can use this gathered information to focus their tools immediately onto the explicit external access portals of a target environment.

The **STRIDE** threat model helps categorize structural weaknesses into six specific areas: **Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege**. For example, applying STRIDE to an authentication page might highlight a severe risk for Elevation of Privilege due to loose access rules. This analysis tells me as an analyst exactly which services are vulnerable, guiding them to test those paths during security checks. What I appreciated about STRIDE was its structure: rather than guessing at what might be wrong with a system, it forces a methodical pass through six distinct failure categories, so that no obvious class of weakness is accidentally skipped simply because it was not the first thing that came to mind.

Additionally, threat actors often leave technical clues or discuss deployment tools on developer forums or public repositories under aliases. Finding this public intelligence allows defenders to tie an identity to specific attack patterns. Instead of searching through generic records, an analyst can look for the exact system behaviors or protocols mentioned in those public posts, quickly locating where a system was accessed. Practicing this kind of profiling on a fictional alias during Mini Project 1 made the idea concrete rather than theoretical, since it meant working through the same patient, evidence-led process a real OSINT investigation would require, rather than simply reading about how such an investigation might be conducted.

Section 3: Week 3 Evidence (Why log analysis is the backbone of security)

Week 3 introduced log analysis using command-line filters like **grep, awk, and sed,** along with automated Bash and Python scripts. Log data provides the objective history of everything that happens on an asset. Analyzing these records is standard practice for identifying active threats and rebuilding timelines during an incident response. Of everything covered so far in the programme, this was the week that felt most directly tied to real analyst work, because logs do not lie about what happened on a system, even when the surrounding narrative is unclear.

Writing automated parsing scripts like security_check.sh and log_analyser.py mimics the core function of an enterprise monitoring platform. While tools like **Splunk** ingest vast streams of network data to organize and alert on specific patterns, a local script performs the same basic logic manually: reading text lines, filtering data with text rules, and counting failures to catch anomalies before they cause widespread impact. Writing those scripts by hand, even small ones, gave me a far better appreciation for what an enterprise platform is actually doing under the surface, since the underlying logic is the same whether it runs on a single laptop or across a corporate network.

During a system investigation, an analyst can isolate specific attack indicators from file transfer services using standard commands. The explicit syntax used to target and filter anomalous entries within a file log is:

```
grep -i "vulnerable" /var/log/vsftpd.log | awk '{print $11}' | sort |
uniq -c | sort -rn
```

This command extracts and summarizes specific error signatures or source information from raw log text. An enterprise alert tool operates identically: it automatically reviews live data lines for matching text strings and triggers a notification whenever the occurrence crosses an established safety limit.

Log filtering is the primary skill required for an entry-level security analyst because configurations can be circumvented, but service records maintain the definitive trace of a breach. Relying on simple summary indicators often misses underlying issues. Reviewing log lines directly allows an analyst to discover successful authentication attempts following brute-force failures, giving them the clear evidence needed to isolate a compromised asset immediately. All of the scripts, command references, and sample log files referenced across these four weeks have been organized and published as my first piece of professional documentation, available at https://github.com/Godswill-star-spec/Onwubiko_Godswill_C--Techrise-weeks1to4.git, which I am treating as the starting record of my practical work rather than a private set of exercises.

Section 4: Week 4 Exploitation (Understanding the attacker's perspective)

Week 4 introduced security testing tools, scanning frameworks, and basic exploitation concepts. Learning to view an environment from an attacker's point of view helps defenders understand how small configuration bugs combine to create complete system vulnerabilities. A defender who only understands administrative checklists will easily overlook non-obvious ways that systems can be breached. This was, in many ways, the week that tied the previous three together, because none of the navigation, intelligence-gathering, or log analysis skills mean much until they are tested against an actual attempt to break into a system.

Exploiting an unpatched service vulnerability can often be accomplished in just a few simple steps using automated frameworks. This ease of entry shows that no organization is too small or insignificant to be targeted by malicious activity. Automated scanning tools on the internet do not look at business size; they look for known, unpatched software versions, turning any unprotected system into an immediate target.

A typical attack path flows from passive intelligence gathering to network scanning, identifying software versions, executing an exploit against a vulnerable service, and confirming control over the target system. A defender can stop this progression at several points. For example, keeping software updated removes the exploit target completely, while a properly configured local firewall blocks unauthorized connections even if an older service is running. Mapping this chain out step by step, rather than only thinking about exploitation as a single event, was what made the defensive side of the lesson click for me, since every stage in that chain represents a separate opportunity to break the attacker's momentum.

The most important lesson from this module is that absolute security does not depend on buying expensive tools. True system safety is determined by its weakest configuration link. A single forgotten, outdated service on an internal system can completely undermine otherwise

excellent network security defenses. It is a humbling lesson, in a way, because it means that good security work is less about acquiring the most advanced tooling and more about consistent discipline applied everywhere, including the parts of a system nobody thinks to check.

Section 5 : The Connection (The link between differing concepts)

Security is not a collection of isolated topics; it is a single continuous practice where each stage builds directly on the previous one. The logical workflow across the first four weeks connects directly as follows, and seeing it laid out this way is what finally made the structure of the whole programme make sense to me:

```
[WEEK 1: FOUNDATION]    --> Learn system navigation and permission
controls.
       |
       ▼
[WEEK 2: INTELLIGENCE]  --> Gather public research and build threat
models.
       |
       ▼
[WEEK 3: EVIDENCE]      --> Filter and parse log lines to find anomalies.
       |
       ▼
[WEEK 4: EXPLOITATION]  --> Think like an attacker to validate
vulnerabilities.
```

In practical security work, these components operate together. Week 1 gives you the ability to navigate a system; Week 2 defines the likely targets; Week 3 extracts the evidence from operational logs; and Week 4 verifies those discoveries by simulating real-world attack behaviors. Removing any piece leaves an analyst unable to defend an environment effectively. I have come to think of it almost as a chain of custody for understanding itself: each week hands a usable skill to the next, and a gap anywhere along that chain quietly weakens everything built on top of it.

Looking ahead, understanding exploitation from Week 4 helps an analyst build smarter monitoring rules in Week 6, because you must know how an attack works to detect it. Similarly, the command-line log analysis skills from Week 3 form the exact foundation required to perform deep forensic investigations on system images later in the program. I expect this pattern to keep repeating throughout the rest of the cohort: each new module will likely depend on something learned earlier, which is itself a useful reminder not to treat any single week's material as disposable once the assignment for it has been submitted.

Section 6: Personal Reflection (what I learned about myself so far)

Working through these core security modules has required a strong focus on technical problem-solving and rapid adaptation. The most challenging concept was mastering data management within automated Python script logic. Writing cleaner code that correctly handles formatting errors without crashing was frustrating, but I overcame this by breaking the script into smaller parts and verifying each feature individually with test data files. Looking back, the

frustration itself was useful, because it forced me to slow down and actually understand what each function was doing instead of copying patterns without thinking them through.

A major turning point occurred during the Week 4 testing labs. For the first few weeks, commands and log files felt like abstract ideas. However, when I executed a test scan against an older service and immediately received a system control prompt, the relationship between configuration choices and real security risk became completely clear. I realized that keeping a system safe requires an active, practical understanding of how components interact. That single moment changed how I read every system afterward, because it is hard to look at an unpatched service the same way once you have personally seen how easily it gives way.

Based on these lab experiences, I want to specialize as a Penetration Tester and Offensive Security Engineer. The process of evaluating an environment, locating hidden administrative flaws, and helping organizations fix their issues before an actual security incident occurs is deeply satisfying. This role allows me to use my background in engineering alongside automated scripting to build resilient, trustworthy systems. Keeping a public record of this early work, including the repository linked in Section 3, matters to me beyond the requirements of the assignment itself, since it marks the point where my learning stopped being private practice and became something I am willing to stand behind as a piece of professional output.